Group 3 - Space Complexity

Members:
Kristy Mok, Ben Chalk, Corina Rivero Montiel, Euan Faller, Joel Cutler, Luke Callen, Sam Ade Fowodu

Software Testing Report

Testing methods and approaches

In order to ensure the game is up to our coding standard, and does not crash or encounter errors, we made use of extensive testing throughout the implementation of the game. We decided that the best approach was to use a hybrid of both automated testing, including unit tests and style tests, as well as manual tests since some of the criteria requirements are difficult to automate, and some also depend on user interpretation in order to measure the effectiveness of what was implemented.

We decided it would be helpful to create a test plan before we started our testing. This enabled us to ensure that all areas of the game were adequately tested, as well as clearly categorising what needed to be done in a manual test and what could be automated. Given the group nature of the project, having clearly defined tests to carry out and report back on was also extremely useful for the delegation of work between us - it meant we wouldn't accidentally end up doing the same tests as each other. It also allowed us to clearly map the requirements to the tests, ensuring clear traceability and full coverage in all areas of the game. To make the test outcomes stand out, successful tests are in GREEN, unsuccessful are in RED and partially successful are in ORANGE highlighting.

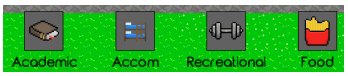
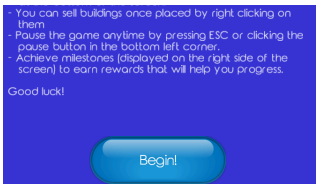
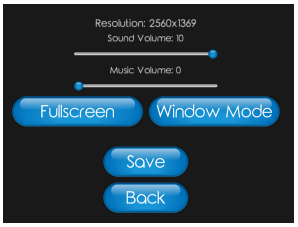
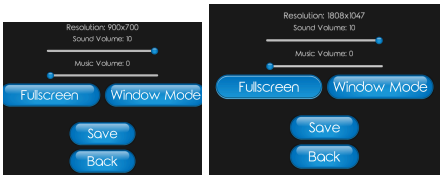
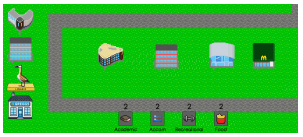
For our automated tests, our approach was to make a separate test file for every code file. This meant that we had modularity in the code and each area could be tested on its own more effectively than having to run the automated tests on everything at once, and kept the test file in the same directory hierarchy as the code files, as this meant that it would be easily accessible for anyone trying to work on an area of the code to be able to find the corresponding test. We also ensured that all aspects of the code were covered and did not miss anything through a systematic approach, using an extensive range of tests such as boundary tests, invalid tests and normal

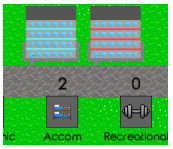
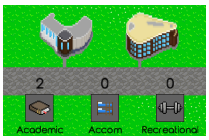
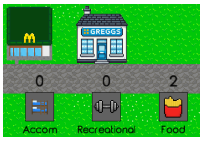
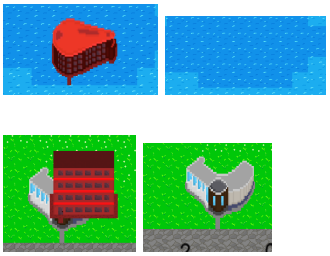
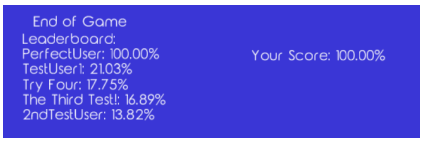
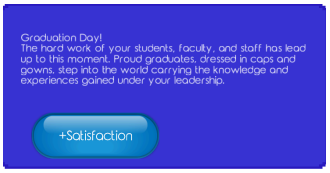
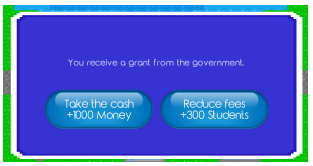
We automated our tests through the use of CI using GitHub actions using well-known Java testing frameworks and tools:

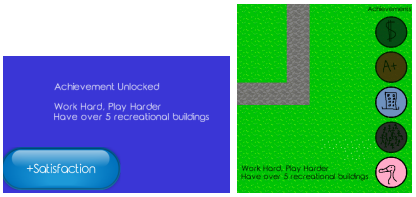
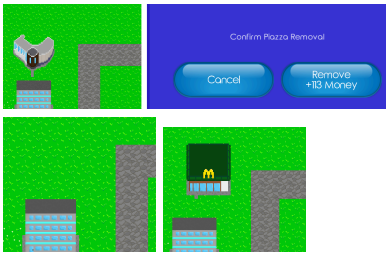
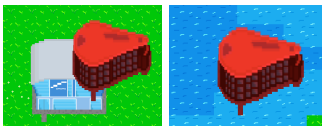
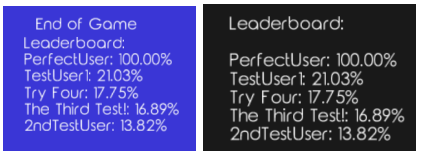
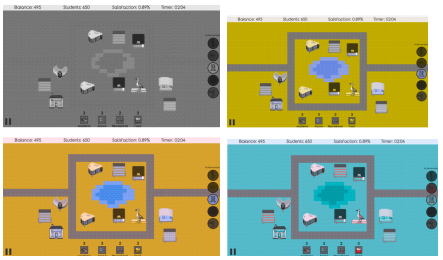
- JUnit - A Java testing framework to write unit tests for our code.
- Mockito - A Java mocking framework that allows us to mock certain Java classes for testing. Importantly for us, com.badlogic.gdx.graphics.GL20, so that we can run the game in headless mode for our tests.
- Jacoco - A code coverage library for Java to check that our tests are reaching all parts of the code.
- Checkstyle - Development tool to help write Java code that adheres to a coding standard. It automates the process of checking Java code This makes it perfect to enforce a coding standard on our project.

Some of the tests proved to be particularly challenging, given the specifications. For example, the NFR_Colour requirement required us to have a colour-blind person present for the testing process, which we did not, so instead we made use of technological solutions, finding a site which can simulate colour blindness and applying it to images of the game: https://www.color-blindness.com/coblis-color-blindness-simulator/.

Other tests were easy to test but harder to demonstrate and document effectively, such as the NFR_FAST_PLACEMENT requirement, which required buildings to be placed in less than 2 seconds. We tested multiple buildings being placed and reported our findings in the test report.

Corresponding requirement ID	Test	Test evidence	Test outcome
FR_BUILDING_TYPE	Every building should have a single purpose such as teaching, food, etc.		There are categories of buildings which can be selected by clicking one of the four buttons
FR_CONTROLS	The system shall be operated by using a standard mouse and keyboard		Allows for mouse or keyboard for various tasks (e.g. ESC key or clicking pause button to pause)
FR_SYSTEM_REQUIREMENTS	The system shall be usable on a big screen with a software lab pc (integrated intel graphics, w11 and 16gb of ram)		Game is functional on the specified device
FR_ACCESSIBILITY	Buttons must have clear descriptions and easy to understand functions		Buttons contain clear and intuitive images and textual label descriptions for further clarity
FR_SOUND	The system should have a sound system integrated with ability to change the sound files		Has game music, modified by altering the path to another sound file
FR_MUTEABLE	There should be a mute button so that the game can be played without any sound playing.		No mute button, but the volume slider can be set to 0, providing the same functionality
FR_SCALING	The system shall be able to scale to small and large screen with low hardware requirements		Automatically able to adjust resolution scaling, with an added fullscreen mode
FR_TIMER	The system shall be able to keep track of the time of the game and end when the timer has finished		When the timer hits 0 and end-of-game screen is shown
FR_MAP	The system must support a map that can fit all required buildings for the player to fulfil objectives		Map easily allows at least 2 of each category of building to be placed

FR_ACCOMODATION_BUILDING	There shall be at least 2 placeable accommodation buildings		There are two distinct accommodation buildings that can be placed
FR_LEARNING_BUILDING	There shall be at least 2 placeable buildings for students to learn in.		There are two distinct buildings for learning that can be placed
FR_EATING_BUILDING	There shall be at least 2 placeable buildings for students to eat in		There are two distinct buildings for eating that can be placed
FR_RECREATIONAL_BUILDING	There shall be at least 2 placeable buildings for recreational activities		There are two distinct buildings for recreational purposes that can be placed
FR_NO_OVERLAP	The system should not allow buildings to be placed on top of one another, or on squares that are off limits (e.g. in a lake)		After a left click on a lake and a building, no building is added. After a left click on another building, there is also no new building added.
FR_LEADERBOARD	The system shall have a leaderboard with the name and score of the top 5 scores		Shows the name and satisfaction score of the top 5 users and the current score
FR_PLANNED_EVENTS	The system will include predictable, fixed events, such as graduation		The graduation event will always occur at the end of the game, boosting student satisfaction
FR_UNPLANNED_EVENTS	The system will include random events such as weather conditions that cannot be predicted		Random events occur and are often presented as choices the user must make
FR_MEASURE_ACHIEVEMENTS	The system will track user data in order to check whether certain goals have been achieved		Has a series of checks based off of stats as to whether the achievement has been achieved

FR_DISPLAY_ACHIEVEMENTS	The achievements that the user has obtained will be clearly displayed to them		Message displayed when achievement is unlocked and there is a textual description and highlighting at the side of the screen, as well as highlighting the achievement once achieved
FR_DELETE_BUILDING	Each of the buildings will have a mechanism to be destroyed and allow for new buildings to be placed in this location		The building can be sold for less than the original price and then a new building can be placed without collision to where the old one was
FR_SHOW_COLLISIONS	If the user is attempting to place a building over another building, it will highlight that it cannot be placed (e.g. by turning red)		Building goes red and does not place if it is attempted to be placed on an invalid location
NFR_EASE_OF_USE	This criteria is fulfilled if users who have never seen can <70% of the time play it without asking questions.	User feedback - 100% could play it without help, but 60% had questions about clarity in aspects of the game	All users were able to successfully play the game without help, though some asked questions about what the metrics meant
NFR_SAVES	The criteria is fulfilled if users can access and see the leaderboard of top scores		Users can access the leaderboard through the end of the game screen, or from the main menu screen
NFR_COLOR	This criteria is met when colour blind users can successfully play the game without needing help		The buildings are still clear through a variety of color blindness conditions. We put the game through red, green, blue and monochrome color blindness filters, and found the game was still accessible.
NFR_FAST_PLACEMENT	In <2 seconds after placing	Placement time tests: 0.14s, 0.28s, 0.22s, 0.21s, 0.14s	Mean placement time = 0.198s, which is far less than 2 seconds